

ARSH ADVANI

United States · 669-609-2927 · arshadvani3@gmail.com · [linkedin.com/in/arshadvani/](https://www.linkedin.com/in/arshadvani/) · github.com/arshadvani3

EDUCATION

University of California, Santa Cruz

Master of Science, Computer Science | GPA: 3.58/4.0

June 2026

Mukesh Patel School of Technology Management and Engineering, NMIMS

Bachelor of Technology, Computer Engineering

June 2024

SKILLS

Languages: Python, TypeScript, JavaScript, C++, SQL

AI / ML: LangGraph, LangChain, RAG, sentence-transformers, PyTorch, HuggingFace, Pydantic, Scikit-Learn

Backend & Web: FastAPI, Node.js, Express.js, Flask, Django, React, GraphQL, WebSockets, REST APIs

Cloud & DevOps: AWS (Lambda, S3, CloudFront), GCP, Docker, Kubernetes, GitHub Actions, CI/CD, OPA

Databases: PostgreSQL, Pinecone, MySQL, MongoDB, Redis, Qdrant, ChromaDB

WORK EXPERIENCE

BigShorts, Software Engineering Intern

June 2025 – September 2025

- Built and owned a production LLM assistant on Azure serving 10,000+ users, architecting the full RAG pipeline with an orchestration loop handling tool routing, context management, and fallback retries at sub-2s p95 latency.
- Engineered a two-stage retrieval-plus-validation layer with structured logging and latency tracing, cutting irrelevant responses 60% while driving prompt and chunking iterations that lowered the production hallucination rate.
- Architected a Node.js and MySQL content backend for the same 10,000+ user product across 6 JWT-authenticated REST APIs at sub-200ms, eliminating N+1 queries and applying denormalization to improve read performance 40%.
- Deployed a serverless AWS Lambda video pipeline converting 500+ uploads to HLS adaptive bitrate streams via FFmpeg and CloudFront CDN, cutting bandwidth consumption 35%.

LTIMindtree, Data Science Intern

January 2024 – June 2024

- Automated enterprise diagram-to-text conversion using LLaVA vision-language models across 200+ workflow diagrams, cutting manual documentation effort by 40% and enabling downstream NLP pipelines on previously unstructured content.
- Benchmarked LLaMA, Mistral, and open-source LLMs across parameter efficiency, latency, and accuracy; reduced hallucination rates 30% through prompt engineering and informed internal LLM adoption strategy.

PROJECTS

AgentMesh - P2P Agent Discovery & Routing Framework [\[GitHub\]](#) | Python, FastAPI, LangGraph, WebSockets, React, TypeScript

- Architected a peer-to-peer agent discovery protocol where agents register capabilities at startup and are ranked at runtime via a 4-factor composite scoring formula (semantic similarity via sentence-transformers, EMA trust, availability, cost), replacing static orchestration with a self-tuning routing layer that shifts weights dynamically per caller signal without pipeline rewrites.
- Built a capability-aware OutputEvaluator driving real EMA trust updates via deterministic grounding checks for structured outputs and LLM-as-judge for open-ended tasks, producing differentiated agent reputation scores that compound across tasks and autonomously shift routing.
- Benchmarked across 5 controlled experiments (routing correctness 5/5, trust convergence, resilience under agent failure) with 176 tests covering routing, trust scoring, circuit breaker, auth, and MCP tool integration; measured 0.3–0.5s fixed mesh overhead over direct LLM latency.

ARGUS - Secure Agentic AI Orchestration Platform [\[GitHub\]](#) | FastAPI, React, OPA, LangGraph, PostgreSQL, Redis, Docker

- Built a full-stack middleware platform (FastAPI, React 18, PostgreSQL/pgvector, Redis, Docker Compose) adding security, observability, and multi-agent orchestration to LLM agents over the Model Context Protocol, integrating 40 tools across GitHub, Jira, Slack, and Google Drive.
- Engineered an Open Policy Agent security gateway authorizing every tool call with role-based, fail-closed policy plus a SHA-256 hash-chained tamper-evident audit log, adding 1.26ms overhead per call against a 100ms budget and passing a 20/20 adversarial policy battery.
- Implemented a LangGraph planner/executor/reviewer orchestrator that decomposes natural-language tasks into concurrent tool-call plans, reaching first action in 2.34s and passing 87/87 tests across a 24-scenario benchmark over 5 runs.

Cloud-Native LLM Inference Service | Python, Kubernetes, Docker, Flask, CUDA, GCP, llama-cpp-python

- Deployed Llama-3.2-3B on Kubernetes across GCP (NVIDIA T4) and Nautilus (Titan RTX) with auto-scaling (1–5 replicas) and 4-bit GGUF quantization (6GB to 2GB), achieving 10x latency reduction (26s to under 3s) via CUDA acceleration.
- Benchmarked cross-platform GPU inference with k6, demonstrating 3x lower latency (1.4s vs 4.0s) and 3.1x higher throughput on Nautilus vs GCP under sustained concurrent load with 100% request success rate.